

Builder Design Pattern

Intent

Separate the **construction** of a complex object from its representation, so the same building steps can produce an object step by step. In practice, the Builder pattern lets you create an object by naming each field you set — `.name(...).email(...).build()` — instead of passing a long, order-sensitive list of constructor arguments.

This implementation is the classic **static nested builder** form: the product (`Customer`) has a private constructor, and a nested `CustomerBuilder` is the only thing allowed to create it.

UML class diagram

Customer	Customer.CustomerBuilder
+ name : String	+ name : String
+ email : String	+ email : String
+ Customer(builder) <<private>>	+ name(String) : Builder
+ getName()	+ email(String) : Builder
+ getEmail()	+ build() : Customer

client -> new CustomerBuilder().name(..).email(..).build()

The code, line by line

```
public class Customer {  
    private String name;  
    private String email;  
  
    private Customer(CustomerBuilder customerBuilder) {  
        this.name = customerBuilder.name;  
        this.email = customerBuilder.email;  
    }  
  
    public String getName() { return name; }  
    public String getEmail() { return email; }  
  
    public static class CustomerBuilder {  
        private String name;  
        private String email;  
  
        public CustomerBuilder name(String name) { this.name = name; return this; }  
        public CustomerBuilder email(String email) { this.email = email; return this; }  
  
        public Customer build() { return new Customer(this); }  
    }  
  
    @Override  
    public String toString() { return super.toString(); }  
}
```

The product fields

```
private String name;  
private String email;
```

- `private` — the outside world cannot set these directly. The **only** way to populate a `Customer` is through the builder. This is what gives the pattern control over construction.

The private constructor

```
private Customer(CustomerBuilder customerBuilder) {  
    this.name = customerBuilder.name;  
    this.email = customerBuilder.email;  
}
```

- **private** — no other class can call `new Customer(...)`. This forces every creation to go through `CustomerBuilder`. It is the same "seal the constructor" idea as in Singleton, but here it channels creation through the builder rather than limiting the count.
- It takes the **builder itself** as its single argument and copies the accumulated values out of it. So the builder acts as a temporary "parameter carrier": you fill it up field by field, then hand the whole thing to the constructor in one shot.

The getters

```
public String getName() { return name; }
public String getEmail() { return email; }
```

- Read-only access to the finished object. There are deliberately **no setters** — once built, a `Customer` is effectively immutable (see **Why immutability** below).

The nested builder

```
public static class CustomerBuilder {
    private String name;
    private String email;
    ...
}
```

- **static nested class** — this matters. A `static` nested class does **not** hold a hidden reference to an enclosing `Customer` instance, which is correct because the builder's job is to exist **before** any `Customer` does. A non-static inner class would require an outer instance first, which is a chicken-and-egg problem here.
- It mirrors the product's fields. These are the values being collected before the object is born.

The fluent setter methods

```
public CustomerBuilder name(String name) { this.name = name; return this; }
public CustomerBuilder email(String email) { this.email = email; return this; }
```

- Each stores one value and then **return this**; . Returning the builder itself is what enables **method chaining** (the "fluent interface"): `builder.name(...).email(...)`. Each call hands you back the same builder so you can immediately call the next setter.
- Note the methods are named `name(...)` / `email(...)` rather than `setName(...)` — a common builder convention that keeps the chain reading like a sentence.

build()

```
public Customer build() { return new Customer(this); }
```

- The terminal step. It calls the private constructor (which it is allowed to do, because the nested class is a member of `Customer`) and passes `this` builder in. The result is the fully assembled, immutable `Customer`.

toString()

```
@Override
public String toString() { return super.toString(); }
```

- This override currently just delegates to `Object.toString()`, so it prints the default `Customer@<hashCode>`. It's a placeholder — a "real" builder demo would usually format the fields here (e.g. `"Customer{name=..., email=...}"`). It does not affect the pattern; it's only what shows up when you print the object.

Why the design decisions

Why a builder at all — what problem does it solve?

Two classic problems with plain constructors:

1. **The telescoping constructor.** With many optional fields you end up with a pile of overloads — `Customer(name)`, `Customer(name, email)`, `Customer(name, email, phone)`, ... — that is painful to write and read.
2. **Order-sensitive, unreadable calls.** `new Customer("Avinash", "a@b.com", "9999", null, true)` — which argument is which? Easy to swap two `String`s by accident and the compiler won't notice. The builder replaces this with **self-documenting, named** calls: `.name("Avinash").email("a@b.com")`. Order no longer matters and each value is labeled.

Why the private constructor?

To **guarantee** the builder is the only path in. If the constructor were public, callers could bypass the builder and construct a half-configured object directly, defeating the point. Sealing it makes the builder mandatory.

Why return this; in each setter?

That single line is the entire mechanism behind the fluent chain. Without it the setters would return `void` and you'd be forced to write:

```
CustomerBuilder b = new CustomerBuilder();
b.name("Avinash");
b.email("a@b.com");
Customer c = b.build();
```

Returning `this` collapses that into one readable expression.

Why is the builder `static`?

Because the builder must be usable **without** an existing `Customer`. A non-static inner class is tied to an outer instance (`outer.new Inner()`), but here there is no `Customer` yet — that's the whole point of building one. `static` breaks that dependency and also avoids leaking a reference to a non-existent enclosing object.

Why immutability (no setters on `Customer`)?

Once `build()` returns, the `Customer` cannot be modified — there are no setters and the fields are private. Immutable objects are inherently **thread-safe** (no one can change them under you), safe to share and cache, and safe to use as map keys. The mutable, "still being configured" state lives only in the throwaway builder; the finished product is frozen.

Manual builder vs. Lombok `@Builder`

This module writes the builder **by hand** to make the mechanism visible. In this same project, Lombok's `@Builder` annotation would generate all of this (the nested builder, the fluent setters, `build()`) automatically. The hand-written version is the teaching version — it shows exactly what `@Builder` produces under the hood.

Execution flow (as run from `main`)

```
DesignPatternsApplication.main
├── SpringApplication.run(...)           boots Spring (not required by the pattern)
├── new Customer.CustomerBuilder()       create an empty builder
│   ├── .name("Avinash Patel")          stores name, returns the builder
│   ├── .email("infinityDoesExist@gmail.com") stores email, returns the builder
│   └── .build()                       new Customer(builder) → copies fields in
│       └── returns an immutable Customer
│           ├── getEmail() → "infinityDoesExist@gmail.com"
│           └── getName() → "Avinash Patel"
```

Relationship to the other Creational patterns

- **Factory / Abstract Factory** decide **which class** to instantiate.
- **Builder** is about **how to assemble one object** that has many parts/fields — it doesn't choose a type, it constructs a single complex instance step by step.
- **Prototype** avoids construction entirely by copying an existing instance.

Use a Builder when an object has several fields (especially optional ones) and you want construction to be readable, order-independent, and to yield an immutable result.

Note: `SpringApplication.run(...)` just boots a Spring context and is unrelated to the pattern; the `Customer` here is built manually, not managed by Spring.